



Name: Rahin Ibne Harun

Id: 23-54177-3

Sec: U

Course: Compiler Design Assignment

Final Lab task -1

Task 1:

Compiler Design (Finalterm Lab 1)

Task 1: Write a C++ program that takes the description of a **Deterministic Finite Automaton (DFA)** and one input string, then checks whether the string is **Accepted** or **Rejected** by the DFA.

Input format

- First line: two integers n and m
 - n = number of states, states are numbered from 0 to n-1
 - m = number of input symbols
- Next m lines: the input symbols
- Next line: integer t, the number of transitions
- Next t lines: current_state input_symbol next_state
- Next line: start state
- Next line: integer f, the number of final states
- Next line: f integers representing final states
- Last line: one input string

Output

Print:

- Accepted if the string is accepted by the DFA
- Rejected otherwise

Sample Input	Sample Output
3 2 a b 6 0 a 1 0 b 0 1 a 1 1 b 2 2 a 1 2 b 0 0 1 2 aab	Accepted

Answer:

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {

    int n, m;

    cout << "Step 1: Enter number of states and symbols : ";

    if (!(cin >> n >> m)) return 0;


    cout << "Step 2: Enter the symbols one by one : " << endl;

    for (int i = 0; i < m; i++) {

        char sym; cin >> sym;

    }

    int stateTable[100][256];

    for(int i = 0; i < 100; i++) {

        for(int j = 0; j < 256; j++) stateTable[i][j] = -1;

    }


    int t;

    cout << "Step 3: Enter number of transitions : ";

    cin >> t;

    cout << "Step 4: Enter transitions (Format: current_state symbol next_state):" << endl;

    for (int i = 0; i < t; i++) {

        int u, v;

        char symbol;

        cin >> u >> symbol >> v;
```

```
    stateTable[u][(int)symbol] = v;  
}
```

```
int startState, f;
```

```
cout << "Step 5: Enter start state : ";
```

```
cin >> startState;
```

```
cout << "Step 6: Enter number of final states: ";
```

```
cin >> f;
```

```
int finalStates[100];
```

```
cout << "Step 7: Enter the final state(s): ";
```

```
for (int i = 0; i < f; i++) {
```

```
    cin >> finalStates[i];
```

```
}
```

```
cout << "\n--- DFA Setup Complete ---" << endl;
```

```
cout << "Enter the string to check then tell accpet or reject: ";
```

```
string input;
```

```
while (cin >> input && input != "exit") {
```

```
    int currentState = startState;
```

```
    bool possible = true;
```

```

for (char c : input) {

    if (stateTable[currentState][(int)c] != -1) {

        currentState = stateTable[currentState][(int)c];

        } else {

            possible = false;

            break;

        }

    }

    bool accepted = false;

    if (possible) {

        for (int i = 0; i < f; i++) {

            if (currentState == finalStates[i]) {

                accepted = true;

                break;

            }

        }

    }

    if (accepted) cout << ">> RESULT: Accepted" << endl;

    else cout << ">> RESULT: Rejected" << endl;

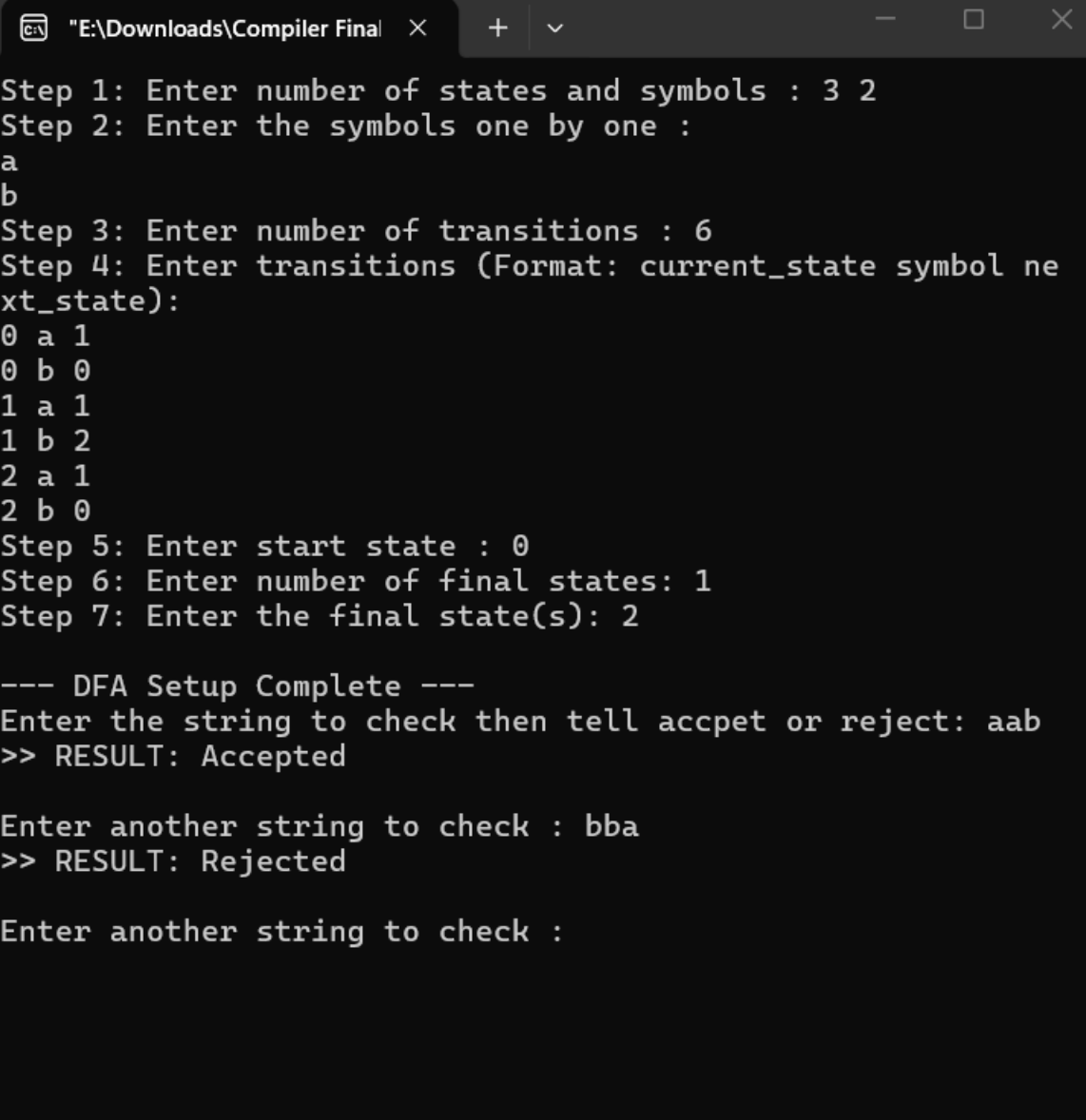
    cout << "\nEnter another string to check : ";

}

```

```
    return 0;  
}
```

Input and output ss:



```
"E:\Downloads\Compiler Final" × + ▾  
Step 1: Enter number of states and symbols : 3 2  
Step 2: Enter the symbols one by one :  
a  
b  
Step 3: Enter number of transitions : 6  
Step 4: Enter transitions (Format: current_state symbol ne  
xt_state):  
0 a 1  
0 b 0  
1 a 1  
1 b 2  
2 a 1  
2 b 0  
Step 5: Enter start state : 0  
Step 6: Enter number of final states: 1  
Step 7: Enter the final state(s): 2  
  
--- DFA Setup Complete ---  
Enter the string to check then tell accpet or reject: aab  
>> RESULT: Accepted  
  
Enter another string to check : bba  
>> RESULT: Rejected  
  
Enter another string to check :
```

Task 2:

Task 2: Write a C++ program that takes the description of a DFA and first checks whether it is a **valid DFA**. If it is valid, then test multiple input strings and print the **state path** and final result for each string.

A DFA will be considered valid only if:

- every state number is within range
- every transition symbol belongs to the given alphabet
- there is not more than one transition for the same (state, symbol) pair
- the start state is valid
- every final state is valid
- for every state and every symbol, there is exactly one transition

Input format

Use the same DFA input format as Task 1, then:

- one line containing integer q, the number of test strings
- next q lines, one input string per line

Output

- If the DFA is invalid, print:
 - Invalid DFA
 - and the reason
- If the DFA is valid, print:
 - Valid DFA
 - then for each input string:
 - the visited path
 - Accepted or Rejected

Sample Input:	Sample Output:
3 2	Valid DFA
a	String: ab
b	Path: 0 -> 1 -> 2
6	Accepted
0 a 1	String: aab
0 b 0	Path: 0 -> 1 -> 1 -> 2
1 a 1	Accepted
1 b 2	String: bbb
2 a 1	Path: 0 -> 0 -> 0 -> 0
2 b 0	Rejected
0	
1	
2	
3	
ab	
aab	
bbb	

Answer:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <map>
```

```

#include <set>

using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<char> symbols(m);
    for (int i = 0; i < m; i++) {
        cin >> symbols[i];
    }
    int t;
    cin >> t;
    map<pair<int, char>, int> trans;
    set<pair<int, char>> checkDuplicate;

    bool valid = true;
    string reason = "";
    for (int i = 0; i < t; i++) {
        int from, to;
        char ch;
        cin >> from >> ch >> to;
        if (from < 0 || from >= n || to < 0 || to >= n) {
            valid = false;
            reason = "State out of range";
        }
        bool found = false;
        for (char s : symbols) {
            if (s == ch) found = true;
        }
        if (!found) {
            valid = false;

```



```

    reason = "Invalid symbol in transition";
}

if (checkDuplicate.count({from, ch})) {
    valid = false;
    reason = "Duplicate transition";
}
checkDuplicate.insert({from, ch});
trans[{from, ch}] = to;
}

int start;
cin >> start;
if (start < 0 || start >= n) {
    valid = false;
    reason = "Invalid start state";
}

string finalStr;
cin >> finalStr;
set<int> finalStates;
for (char c : finalStr) {
    int f = c - '0';
    if (f < 0 || f >= n) {
        valid = false;
        reason = "Invalid final state";
    }
    finalStates.insert(f);
}

for (int i = 0; i < n; i++) {
    for (char s : symbols) {
        if (!trans.count({i, s})) {
            valid = false;

```

```
reason = "Missing Something";  
    }  
}  
}
```

```
if (!valid) {  
    cout << "Invalid DFA\n";  
    cout << reason << endl;  
    return 0;  
}
```

```
cout << "Valid DFA\n";  
int q;  
cin >> q;
```

```
while (q-- > 0) {  
    string input;  
    cin >> input;  
    int current = start;  
    cout << "String: " << input << endl;  
    cout << "Path: " << current;  
    for (char ch : input) {  
        current = trans[current][ch];  
        cout << " -> " << current;  
    }  
    cout << endl;  
    if (finalStates.count(current)) {  
        cout << "Accepted\n";  
    } else {  
        cout << "Rejected\n";  
    }  
}
```

